

tmux documentation

tmx Shell Integration — Operator Guide

Revision: 1 **Last modified:** 2026-05-22T14:30:00Z **Authority:** vasic-digital tmux project
Maintainer: milosvasic **Scope:** Operator install / uninstall / troubleshoot guide for the v1.0.9
`tmx-shell-init.sh` interactive-prompt feature

1. Overview

`scripts/tmx-shell-init.sh` is the project-owned shell-init script you source from your `.bashrc` / `.zshrc`. It replaces the hand-pasted snippet that operators previously copied into every rc file (the old snippet drifted silently, blocked SCP / IDE shells with a hung `read -r`, and forgot the operator's last cwd).

The new init script:

- prompts only on **interactive TTY** shells (skips on SCP, rsync, IDE pipes, cron, non-interactive subshells);
- defaults to **SKIP** (bare shell, no tmx) when the operator presses Enter or types `default`;
- on any other name, runs `tmx attach -t NAME` and falls back to `tmx new -s NAME` if no such session exists;
- on `tmx new`, the wrapper recalls the session's last cwd via `scripts/tmx-state-bin` and starts the pane there (see [docs/guides/tmx-state.md](https://docs.guix.gnu.org/en/manual/guix-1.10/html_node/terminal.html#tmx-state)).

Project authority spec: `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md §4.A + §5`.

2. Prerequisites

- A POSIX shell: bash 3.2 (macOS default), bash 5+, zsh 5.8+, or dash.
- tmx on `$PATH`. After `bash scripts/setup.sh GREEN`, the snippet already prepends `scripts/` to `PATH` for you.
- The Go-built state daemon at `scripts/tmx-state-bin` (also built by `setup.sh`). Without it, `tmx new` still works, the cwd just won't be restored.

3. Installation

3.1 Recommended path (use the generated snippet)

```
cd ~/Projects/tmux
# or wherever you cloned this repo
bash scripts/setup.sh
# setup.sh GREEN summary prints the exact line to copy.
```

`setup.sh` writes the snippet into `~/ .bashrc` AND `~/ .zshrc` automatically on a fresh install. Open a new terminal — that's it.

3.2 Manual path (if you maintain your own rc)

Append the following block to **both** `~/ .bashrc` and `~/ .zshrc`, with `/Users/milosvasic/Projects/tmux` replaced by the absolute path that `pwd` prints while you stand in the project root:

```
# — vasic-digital optimized tmux
VDIGITAL_TMUX_DIR="/Users/milosvasic/Projects/tmux/scripts"
if [ -x "$VDIGITAL_TMUX_DIR/tmx" ]; then
    export PATH="$VDIGITAL_TMUX_DIR:$PATH"
fi
[ -r "$VDIGITAL_TMUX_DIR/tmx-shell-init.sh" ] && .
    "$VDIGITAL_TMUX_DIR/tmx-shell-init.sh"
# — end vasic-digital optimized tmux
```

Reload the rc and verify:

```
exec $SHELL -l
which tmx # → /Users/milosvasic/
    Projects/tmux/scripts/tmx
type tmx-shell-init.sh 2>/dev/null # not on PATH itself —
    sourced via `.`
```

3.3 Note for macOS Apple Silicon operators

On macOS the project default cwd is `~/Projects/tmux`. If you cloned elsewhere, replace `/Users/milosvasic/Projects/tmux` everywhere with your absolute path (run `pwd` while standing in the repo root to read the value).

4. Behavior contract

The init script returns silently (exit 0) without prompting in these situations:

Condition	Why
<code>\$TMUX</code> already set	already inside a tmux session
<code>[! -t 0]</code>	
<code>\$TMX_SKIP</code> non-empty	operator opt-out for this shell only
<code>tmx</code> not on <code>\$PATH</code>	graceful degradation
EOF on stdin (Ctrl-D at prompt)	bare shell
Empty input or the literal <code>default</code>	bare shell

The script prompts (and acts on the answer) only when none of the above guards trigger.

Invalid names print an error and return 1:

- length zero or > 64 characters;
- any character outside `[A-Za-z0-9_.-]`.

On a valid name it runs:

```
exec sh -c 'tmx attach -t "$1" 2>/dev/null || exec tmx new -s "$1"' tmx-shell-init "$session_name"
```

exec replaces the current shell with the tmux client so that pressing Ctrl-b d (detach) returns you cleanly to the parent shell.

5. Worked examples

Example 1 — New interactive login, attach to existing 'work'

```
$ ssh milosvasic@nezha.local
# .bashrc is sourced → tmx-shell-init.sh prompts:
[tmx] Enter session name (blank or "default" = bare shell): work
# If 'work' exists → attached to it; pane cwd restored to
    wherever you
#   were when you last detached.
# If 'work' did NOT exist → tmx new -s work -c <recalled-pwd> is
    run.
```

Example 2 — SCP from another machine never hangs

```
# On your workstation:
$ scp report.pdf milosvasic@nezha.local:/tmp/
report.pdf                                100% 1.2MB   3.0MB/s
    00:00
# scp succeeds immediately — tmx-shell-init.sh sees the non-TTY
    stdin
# and silently returns 0 before any prompt is printed.
```

Example 3 — Opt out for this shell only

```
$ TMX_SKIP=1 bash -l
# .bashrc runs, but tmx-shell-init.sh exits early (TMX_SKIP non-
    empty)
# You land directly in a bare bash shell, no tmx, no prompt.
$ unset TMX_SKIP
$ exec $SHELL -l
# Back to the prompt-on-login behaviour.
```

Example 4 — Inside a nested tmux pane

```
$ tmx new -s outer                # creates session 'outer'
# inside the pane:
$ bash -l                        # spawns a sub-shell
```

```
# .bashrc runs again, but $TMUX is set, so tmx-shell-init.sh is
    silent.
# No prompt, no nesting — Ctrl-b d still detaches the outer
    session.
```

Example 5 — Manual probe without installing

```
# Run the script under a sub-shell with stdin redirected from /
    dev/null
# (mimics the SCP non-TTY case).
$ bash -c './Users/milosvasic/Projects/tmux/scripts/tmx-shell-
    init.sh' </dev/null
$ echo $?
0
# Exit 0 with no output = non-TTY guard fired as designed.
```

6. Troubleshooting

Symptom	Diagnosis	Fix
tmx-shell-init.sh: command not found	The PATH export wasn't loaded (rc edit didn't take)	<code>exec \$SHELL -l</code> to reload, or check the snippet sits BEFORE <code>if [-z "\$PS1"]</code> early-exit guards
Prompt never appears on interactive login	tmx missing from PATH, or shell isn't sourcing your rc	Run <code>which tmx</code> ; if empty, re-run <code>bash scripts/setup.sh</code>
Prompt appears, but Enter does nothing	Your terminal sends <code>\r\n</code> ; the script accepts both (POSIX <code>read -r</code>)	Type letters; an empty answer maps to SKIP by design
[tmx] invalid session name '...' on a normal name	The name contains <code>;</code> , <code>,</code> , <code>/</code> , or <code>.</code>	Rename — only <code>[A-Za-z0-9_.-]{1,64}</code> is accepted
Hangs forever in SCP/rsync	The init script was modified or you're using the legacy hand-pasted snippet	Re-run <code>setup.sh</code> ; the v1.0.9 script tests <code>-t 0</code> BEFORE prompting
Restored cwd is wrong (always \$HOME)	<code>tmx-state-bin</code> not on PATH OR state file <code>~/.tmx/state.json</code> unreadable	<code>ls -la ~/.tmx/state.json</code> ; <code>chmod 600</code> if needed; see tmx-state guide

7. Uninstall

7.1 Standard uninstall (v1.0.11+)

The cleanest path is the dedicated operator entry point:

```
cd ~/Projects/tmux
bash scripts/uninstall.sh
```

This delegates to `setup.sh --uninstall` (single source of truth) and performs ALL of:

1. Strip the `—— vasic-digital optimized tmux ——` fenced block from `~/.bashrc` and `~/.zshrc`.
2. Strip any LEGACY pre-v1.0.9 unfenced `if command -v tmx ...; fi` snippet (operators who hand-pasted the original snippet).
3. Remove `~/.tmux.conf` (only if generated by `setup.sh`).
4. Remove `scripts/tmx` (generated wrapper).
5. Remove `scripts/tmx-shell-init.sh` (generated v1.0.9+ shell init).
6. Remove `scripts/tmx-state-bin` (v1.0.9+ Go binary).

7.2 Also purge per-session state (optional)

By default the uninstall preserves `~/.tmx/` (per-session last-pwd records) per §9 zero-risk-data-safety. To purge:

```
bash scripts/uninstall.sh --purge-state
```

7.3 Equivalent invocation

```
bash scripts/setup.sh --uninstall          # same as scripts/
      uninstall.sh
PURGE_STATE=1 bash scripts/setup.sh --uninstall  # same as --
      purge-state
```

7.4 Clean-slate before reinstall

`setup.sh` automatically runs the uninstall logic at step 0 (silent) before re-installing — so an operator running `bash scripts/setup.sh` on a host that already had v1.0.9 installed gets a clean state without needing to manually uninstall first. v1.0.11+ behaviour.

7.5 Hand-removal (last resort)

If you do not have access to `setup.sh` or `uninstall.sh` and need to remove the snippet manually:

Open `~/.bashrc` and `~/.zshrc`; delete the block between `# —— vasic-digital optimized tmux ——` and `# —— end vasic-digital optimized tmux ——`. Then `rm scripts/tmx scripts/tmx-shell-init.sh scripts/tmx-state-bin ~/.tmux.conf` (only the last one if you didn't keep your own; check for `vasic-digital optimized tmux` configuration marker in the file first).

8. Cross-references

- [docs/guides/tmx-state.md](#) — Go state daemon CLI + state file schema
- [docs/guides/tmx-ssh-dispatch.md](#) — `ssh host-tmx <session> setup`
- [docs/manual/tmx-shell-integration.md](#) — end-user master manual
- [docs/scripts/tmx-shell-init.md](#) — §11.4.18 script companion
- Spec: [docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md](#) §4.A + §5

9. Anti-bluff (§11.4)

This feature is covered by Layer 3 tests `28_default_skip.sh`, `29_default_skip_blank.sh`, `30_non_tty_skip.sh`, `35_session_name_validation.sh`, `37_nested_tmux_skip.sh`, and the Layer 4 paired mutation M20 (strips the `-t 0` guard, asserts test 30 FAILs). Every PASS captures positive runtime evidence (no tmux process spawned in skip-paths; named-session created in valid-name paths).

10. Last verified

2026-05-22 on Darwin arm64 (macOS 15.x, bash 3.2 / zsh 5.9, tmux 3.6a) with bash
`scripts/setup.sh --verify-only` GREEN.